

ARCHITECTURE REFERENCE

ComplianceWatch

Regulatory change, turned into cited, dated work
for every Indian small business

ComplianceWatch watches Indian regulators for rule changes, decides which changes apply to one specific business, and turns each into an obligation with a due date, steps and the exact clause it comes from. The launch vertical is GST (CBIC notifications and circulars); FSSAI is the second regulator.

This document is the reference for how the system is built: requirements, high- and low-level design, the AI layer, data, APIs, code structure, operations, cost and roadmap. It is written for engineers joining the project and for reviewers who want to see how the pieces fit.

Author

Himanshu Dixit

Version

1.0 · 27 September 2026

Code

[github.com/himanshudixit2002/
compliancewatch](https://github.com/himanshudixit2002/compliancewatch)

Portfolio

[himanshu-portfolio-eight-flame](#)
[.vercel.app](#)

Stack at a glance

Web and channels	Next.js 16, TypeScript, Tailwind; WhatsApp Business Cloud API (Hono bot); email via SES
Services	Python 3.12, FastAPI, Pydantic v2, SQLAlchemy 2, Alembic; 10 services in one uv workspace
Orchestration and events	Temporal workflows; Kafka (Redpanda locally) with a transactional outbox
Data	PostgreSQL 16 with pgvector and full-text search; Redis 7; S3-compatible object store
AI	Claude (extraction, judgement) behind an in-house LLM gateway; bge-m3 or Voyage embeddings; cross-encoder reranking; Langfuse tracing; eval harness in CI
Quality	mypy –strict, tsc –strict, ruff, import-linter, pytest + Hypothesis, vitest, gitleaks; 80% coverage floor

How to read the numbers. Figures marked ^T are *design targets* the system is built to meet and is tested against; they are not measurements from production. The [implementation status](#) page lists what is built today.

Contents

Implementation status	4
1 Problem, product and requirements	5
1.1 The problem	5
1.2 Four commitments	5
1.3 Personas and scope	5
1.4 Functional requirements	5
1.5 Non-functional requirements	6
2 High-level architecture	6
2.1 Components	7
2.2 The two flows the design serves	8
3 Domain model	8
3.1 Bounded contexts	8
3.2 Entities	8
3.3 Invariants enforced in code	9
3.4 Three-valued applicability	9
3.5 Lifecycles	9
4 Low-level design	10
4.1 Services	10
4.2 Events	10
4.3 Fan-out for one published rule	11
5 AI layer	11
5.1 Models by task	11
5.2 Layered retrieval	12
5.3 Extraction pipeline	12
5.4 Evaluation	13
6 Data and APIs	13
6.1 Stores	13
6.2 Key tables	14
6.3 Public REST API (v1)	14
6.4 First partner: SmartShelfKart	15
7 Code design	15
7.1 Layers inside every service	15
7.2 Core protocols (from the domain kernel)	15
7.3 Patterns and where they are used	16
7.4 Repository layout	16
7.5 Technology choices	16
7.6 Ownership	17

8	Security, deployment and operations	17
8.1	Security and privacy	17
8.2	Deployment profiles	18
8.3	Service levels and alerts	18
8.4	Testing	18
8.5	Running cost	19
9	Roadmap, decisions and risks	19
9.1	Roadmap	19
9.2	Decision log	19
9.3	Risks	20
9.4	Glossary	20

Implementation status

The repository is built phase by phase against this reference. Each row names the part of the design and where it stands on 27 September 2026.

Area	Status	Notes
Monorepo, service template, CI gates	Built	uv + pnpm/Turborepo workspace; 10 FastAPI services with health, readiness and ping routes; per-service Alembic schemas; Docker Compose dev stack; GitHub Actions; pre-commit hooks
Domain kernel	Built	~2,300 lines, standard library only: typed ids, value objects, three-valued predicate algebra, status machines, protocols, event envelope; property-based tests with Hypothesis
Ontology v0 (GST)	Built	Attribute definitions in YAML with loader and validator (package <code>ontology</code> v0.1.0)
Architecture decision records	Built	ADR-001 to 006 written; 007 to 011 recorded as stubs
LLM gateway	In progress	Skeleton with a deterministic fake provider, cost ledger and routing settings
Regulatory pipeline (crawl, parse, extract, review)	Planned	Phase 1
Applicability engine, obligations, notifications, Q&A	Planned	Phase 2
CA dashboard, partner API, eval dashboard	Planned	Phase 3
Second regulator (FSSAI), Hindi UI	Planned	Phase 4

1 Problem, product and requirements

1.1 The problem

Small businesses learn about rule changes late, usually from their chartered accountant (CA) or from a penalty notice. Regulators publish changes as PDFs and portal notices spread across many sites, with no view of what applies to a particular business. Legal-tech that solves this is priced and built for enterprises.

A chat window over regulator PDFs does not solve it either. It answers “what does this document say”, not “does this apply to me, from when, and what must I do”. Enterprise AI pilots through 2026 mostly failed for the same reasons: no connection to the user’s real data, no memory across weeks, and no way to tell whether a retrieved document is still in force.

1.2 Four commitments

Every design decision in this document serves one of these.

1. **Applicability, not retrieval.** Every rule carries jurisdiction, effective dates, what it supersedes and a structured predicate over business attributes. The engine answers “applies to you from 1 October” rather than “here is a document”.
2. **Acts, not chats.** The output is an obligation with a due date, steps, the evidence to keep and a reminder on WhatsApp or email.
3. **Trust you can see.** Every obligation and answer cites the exact clause. No rule reaches a customer without human approval.
4. **Memory.** The system knows what a business has already done, what changed since and what is due next.

Out of scope for v1: filing returns, payments, legal opinions, and any action on a regulator’s portal on the user’s behalf.

1.3 Personas and scope

Persona	Who	Needs	Phase
Owner-operator	Restaurant, trader, small manufacturer; 1–50 staff	“What do I do and by when”, in plain language, on WhatsApp	2
Practising CA	Manages 50–500 client businesses	One view of changes across clients, bulk notices, audit trail	2–3
Compliance lead	Mid-size company, 100–1,000 staff	Task assignment, evidence, exportable audit log, SSO	3
Regulatory analyst	Internal domain team	Approve extracted rules fast, source side by side	1
Partner developer	ERP, billing and inventory vendors (first: SmartShelfKart)	REST API, webhooks, sandbox	3

1.4 Functional requirements

Each feature has an acceptance criterion that becomes an automated check or a review gate.

ID	Feature	Requirement and acceptance criterion	Phase
F1	Source monitoring	Crawl each registered source on schedule; detect new or changed documents by SHA-256; never lose a document. <i>100% of new CBIC notifications detected within 6 h over 30 days</i> ^T	1

ID	Feature	Requirement and acceptance criterion	Phase
F2	Normalisation	PDF, HTML and scans to clause-level text with stable clause ids; tables kept; OCR fallback. $\geq 98\%$ text fidelity on a 100-document sample ^T	1
F3	Rule extraction	LLM emits a schema-validated rule candidate with predicates, dates, supersession links and citations. $\geq 90\%$ approved without edits by month 2 ^T	1
F4	Human review	Source beside extracted rule; approve, edit, reject; two-person rule for high-impact tags. <i>No rule published without an approver in the audit log</i>	1
F5	Versioned rulebook	Versions with effective_from, effective_to and supersedes; answers “which rules were in force on date D”	1
F6	Business profile	Guided onboarding with GSTIN pre-fill; attributes drawn from the shared ontology. <i>60% finish in ≤ 10 min</i> ^T	2
F7	Applicability engine	Deterministic predicate evaluation; LLM judgement only for free-text conditions, with confidence. <i>Precision $\geq 95\%$, recall $\geq 90\%$</i> ^T	2
F8	Obligations	Due date, steps, evidence type and clause citation for every applicable rule	2
F9	Notifications	WhatsApp and email with batching, quiet hours and dedupe per change per business. <i>Zero duplicates</i>	2
F10	Grounded Q&A	Answers only from published rules, with citations; “not covered” otherwise. <i>Grounded-answer rate $\geq 97\%$</i> ^T	2
F11	Tracking and audit	Status, assignee, evidence, comments, immutable audit log	2–3
F12	CA multi-client view	Changes by client, bulk actions, white-label digest	3
F13	Partner API	OAuth client credentials, signed webhooks, sandbox tenant	3
F14	Admin console	Sources, tenants, flags, LLM budgets, audited read-only impersonation	1–3
F15	Evals	Eval suite on every prompt or model change; blocks merge below thresholds	1–3

1.5 Non-functional requirements

The system must never publish an uncited or unreviewed rule, never lose a regulator document, and fan one change out to 100,000 businesses within an hour.

Area	Target
Scale	Year 1: 10,000 businesses, 500 CA firms, 20 sources. Year 3: 500,000 businesses, 200 sources, without re-architecture ^T
Fan-out	One published change evaluated for 100,000 businesses in under 60 min; idempotent and resumable ^T
Latency	Profile save p95 < 300 ms; single-business recompute < 2 s; grounded Q&A p95 < 4 s ^T
Freshness	Regulator publish to review queue < 6 h ^T
Availability	99.9% monthly for product APIs; the pipeline may lag but never drops a document ^T
Durability	Append-only documents, rulebook and audit log; RPO 15 min, RTO 1 h ^T
Security	Row-level security on every tenant table; TLS 1.2+; AES-256 at rest; audit log kept 7 years
Privacy	India’s DPDP Act 2023: purpose-limited collection, recorded consent, deletion within 30 days, data in India
Cost	LLM spend under INR 15 per active business per month; alarms at 80% of budget ^T
Extensibility	A new regulator adds an adapter, a parser profile and ontology terms, with no change to core services

2 High-level architecture

Two halves share one platform. The **regulatory pipeline** (bottom of Figure 1) turns regulator documents into reviewed, versioned rules. The **product services** (top) read only published rules and never call the pipeline directly.

The applicability engine joins the two.

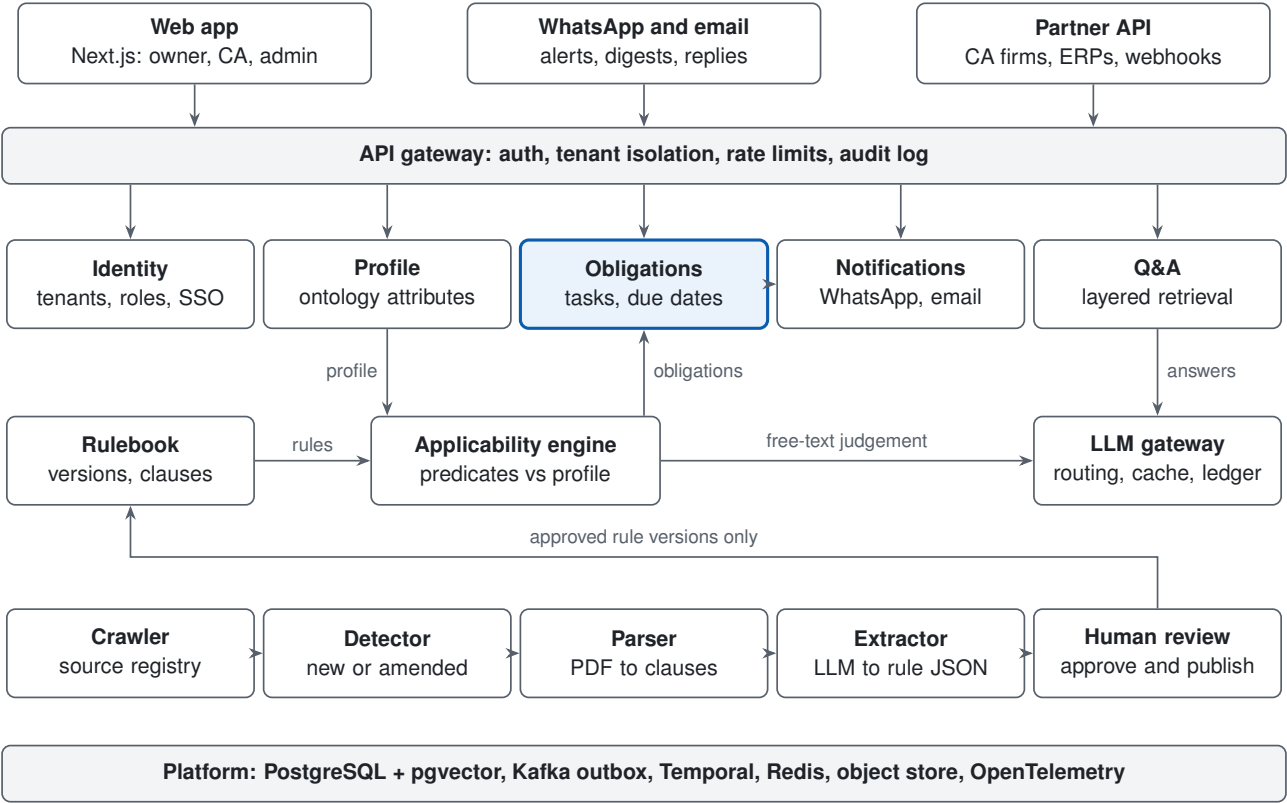


Figure 1: System map. Only an approved rule version reaches the rulebook; product services read the rulebook and never the pipeline.

2.1 Components

Component	Responsibility
API gateway	OIDC token validation, tenant resolution, rate limits, request audit, routing
Identity	Organisations, users, roles, plans, SSO and partner API keys
Profile	Attribute store per business in the shared ontology vocabulary; emits <code>profile.updated</code>
Applicability engine	Evaluates rule predicates against profiles; LLM judgement for free-text conditions; stores every decision with its reasons
Obligations	Creates, tracks and audits obligations; schedules due-date reminders
Notifications	Channel adapters, batching, dedupe, quiet hours, delivery receipts
Rulebook	Published rule versions, clauses, embeddings, entity index, as-of queries
Q&A	Layered retrieval (Section 5.2) with cited answers or an explicit refusal
LLM gateway	Provider routing, prompt registry, semantic cache, cost ledger, PII scrubbing, failover
Crawler	Source registry, scheduled fetch, hash diff, raw storage
Detector	Classifies each document as new, amended, superseding or irrelevant; links it to earlier documents
Parser	Layout-aware PDF and HTML parsing to clauses with stable references; OCR fallback
Extractor	Schema-validated rule candidates with verified citations
Review	Analyst workbench; approval publishes a rule version

2.2 The two flows the design serves

A regulator publishes a change. The crawler stores the raw file and emits `document.discovered`. The detector classifies it and links earlier documents it amends. The parser emits clauses; the extractor emits a rule candidate with citations. An analyst approves it, which publishes a rule version and emits `rule.published`. The engine evaluates every business whose profile passes the rule's coarse filters, creates obligations, and notifications sends one card per business, deduplicated on (rule version, business, channel).

A business changes its profile. The profile service emits `profile.updated`. The engine re-evaluates only that business against the current rulebook, closes obligations that no longer apply with a reason, and opens new ones.

Both flows run as Temporal workflows: durable, retried, resumable, and visible step by step in the admin console.

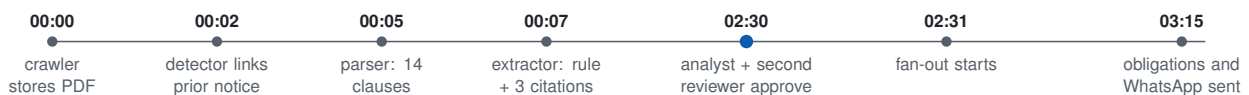


Figure 2: Illustrative timeline for one GST notification. Human review is the longest step, so the review workbench's speed sets the freshness target.

3 Domain model

The core idea is one shared vocabulary, the **ontology**, in which both rule predicates and business profiles are written. Applicability is the evaluation of one against the other. Every other entity exists either to get a regulator document into that form with an audit trail, or to turn the result into work.

3.1 Bounded contexts

Four contexts, each owning its tables and emitting events. References across contexts are by id only, with no cross-context foreign keys.

- **Identity**: tenants, users, roles, API keys.
- **Regulatory**: sources, documents, clauses, rule candidates, rules, rule versions, reviews.
- **Business**: profiles and their versions, the ontology.
- **Work**: applicability decisions, obligations, notifications, evidence.

3.2 Entities

Entity	Context	Key fields and notes
Tenant	Identity	id, name, plan, region. A CA firm is a tenant that manages many businesses
BusinessProfile	Business	tenant_id, attributes keyed by ontology attribute, version. Versioned so every decision can be replayed
Ontology attribute	Business	key, type, allowed values, definition, source (GSTIN lookup or user). Examples: registration_type, turnover_band, state_codes, business_category
Source	Regulatory	regulator, url, adapter type, cadence, last_seen_hash
Document	Regulatory	source, raw_uri, sha256, doc_type, published_at. Append-only; links to documents it supersedes
Clause	Regulatory	document, clause_ref, text, embedding, page. clause_ref is the citation target
ClauseEntity	Regulatory	clause, entity type (notification no., section, HSN code, form), value. Powers multi-hop retrieval (Section 5.2)

Entity	Context	Key fields and notes
RuleCandidate	Regulatory	payload, model, prompt version, confidence, status. Never visible to customers
Rule / RuleVersion	Regulatory	A rule is identity across amendments; a version holds title, summary, predicates, obligation template, effective period, supersedes, citations, approvers
Predicate	Regulatory	attribute, operator, value, optional free_text. Deterministic unless free_text is set
ReviewTask	Regulatory	candidate, assignee, decision, edits as a diff. Two rows for high-impact rules
ApplicabilityDecision	Work	business, rule version, result, confidence, evaluated predicates, profile version. Stored for every evaluation
Obligation	Work	business, rule version, decision, steps, due_at, status, assignee, evidence
Notification	Work	obligation, channel, dedupe_key = hash(rule version, business, channel), status

3.3 Invariants enforced in code

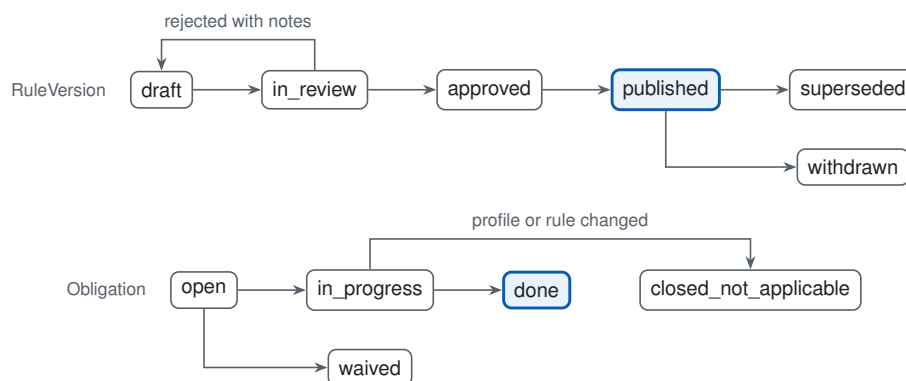
- A rule version cannot be published without at least one citation that resolves to a clause and at least one approver.
- A predicate's attribute must exist in the ontology when it is written; unknown attributes are rejected.
- An obligation always references the exact rule version and profile version it was computed from.
- Documents and clauses are append-only; corrections create new rows.
- Every Business and Work row carries `tenant_id` and is covered by row-level security.

3.4 Three-valued applicability

Predicate evaluation returns one of `applies`, `not_applicable` or `unsure`. A structured predicate whose attribute is missing from the profile is `unsure`; a free-text predicate is always `unsure` until judged. Composites follow Kleene's strong logic: any `not_applicable` decides an `AllOf`, any `applies` decides an `AnyOf`, and otherwise any `unsure` makes the result `unsure`. An `unsure` result never notifies a customer; it asks the business a question or goes to review.

```
# illustrative use of the kernel's predicate algebra
rule = AllOf((
  Predicate("registration_type", Operator.EQ, "regular"),
  Predicate("aggregate_turnover_inr", Operator.GTE, 5_00_00_000),
  Not(Predicate("business_category", Operator.IN, ("government", "sez_unit"))),
))
evaluate(rule, profile) # -> Applicability.APPLIES | NOT_APPLICABLE | UNSURE
```

3.5 Lifecycles



Illegal transitions raise at runtime; the transition tables live in the domain kernel and are tested exhaustively.

4 Low-level design

Ten deployable services, each with one owning team, its own Postgres schema, a REST contract for queries and Kafka events for state changes. Services never read another service's tables. Every pipeline step is a Temporal activity, so retries, timeouts and resumption are declared rather than hand-written.

4.1 Services

Service	Core logic	Failure handling
<code>pipeline</code> (crawler, detector, parser, extractor)	Fetch through a <code>SourceAdapter</code> ; SHA-256 diff; store raw in the object store; classify with a small model (LLM few-shot until labels exist); layout-aware parsing with OCR fallback; structured-output extraction with citation checks	Retry with backoff; alert after 2 missed cadences; parse failures become manual tasks; invalid JSON retried once with the validation error, then routed to review as "extraction failed"
<code>rulebook</code>	Versions, supersession graph, clause index (vector, full-text, entities), as-of queries; review workbench API	Index rebuilds are idempotent; supersession cycles rejected at write; approval and publish in one transaction via outbox
<code>profile</code>	Validates attributes against the ontology; versions every change; GSTIN pre-fill through an anti-corruption adapter	Lookup provider down: user continues manually and is asked to verify later
<code>applicability-engine</code>	Coarse filter by regulator and attribute index, then per-business predicate evaluation; free-text predicates judged by the LLM with confidence; every decision stored with its evaluated predicates	Fan-out is a Temporal workflow with a checkpoint every 1,000 businesses; confidence below 0.8 goes to review and never notifies
<code>obligation</code>	Builds obligations from the rule's template; due dates; reminders; closes obligations when a decision flips	Idempotent on (business, rule version); append-only audit log
<code>notification</code>	Dedupe by key; digest batching; quiet hours; templates per channel and language	Retry then fall back to the second channel; receipts reconcile status
<code>qa</code>	Layered retrieval (Section 5.2); profile-aware phrasing; cited answers or refusal	Nothing above threshold: answer "not covered" and log the question for the eval set
<code>llm-gateway</code>	Model per task; semantic cache; token budgets per tenant and feature; circuit breaker per provider; versioned prompts	Primary provider down: fail over; budget exhausted: queue non-urgent work
<code>eval</code>	Golden sets, metric runs, drift reports	A failed threshold blocks the merge or pages the owning team
<code>identity</code>	Maps OIDC claims to roles; partner API keys; plan limits	IdP unavailable: existing sessions continue, new logins fail closed

4.2 Events

Event	Producer	Consumers
<code>document.discovered</code>	<code>pipeline</code>	<code>pipeline</code> (detector)
<code>document.parsed</code>	<code>pipeline</code>	<code>pipeline</code> (extractor)
<code>rule.candidate.created</code>	<code>pipeline</code>	<code>rulebook</code> (review queue)

Event	Producer	Consumers
<code>rule.published</code> , <code>rule.superseded</code>	rulebook	applicability-engine, qa (cache), partners via webhook
<code>profile.updated</code>	profile	applicability-engine
<code>applicability.decided</code>	applicability-engine	obligation
<code>obligation.created</code> , <code>.due_soon</code> , <code>.closed</code>	obligation	notification, partners via webhook
<code>notification.sent</code> , <code>.failed</code>	notification	obligation (timeline), eval (delivery metrics)

Contract rules. Topics are named `object.verb` in the past tense. Every payload is versioned JSON with a schema in `packages/contracts` and carries `event_id`, `occurred_at`, `tenant_id`, `correlation_id` and `causation_id`. Producers write to an outbox table in the same transaction as their state change, and a relay publishes to Kafka, so there are no lost or phantom events. Consumers are idempotent on `event_id`; each has a dead-letter topic that the admin console can replay after a fix.

4.3 Fan-out for one published rule

1. `rule.published` starts a `FanOutWorkflow(rule_version_id)`.
2. A coarse SQL filter on indexed profile attributes (regulator, registration type, states) returns candidate businesses; a rule that needs `turnover_band ≥ 5cr` never scans businesses below it.
3. Candidates are processed in batches of 1,000 as separate activities; each batch evaluates predicates in-process with no network call unless a free-text predicate needs judgement.
4. Decisions and obligations are written with idempotency keys, so a crashed batch is simply re-run.
5. Notifications are emitted per business, and a CA firm receives one digest covering all its affected clients.

5 AI layer

The LLM does three narrow jobs: extract rules from regulator text, judge free-text conditions, and phrase grounded answers. Every other decision is deterministic code over structured data, which keeps cost predictable and every output testable.

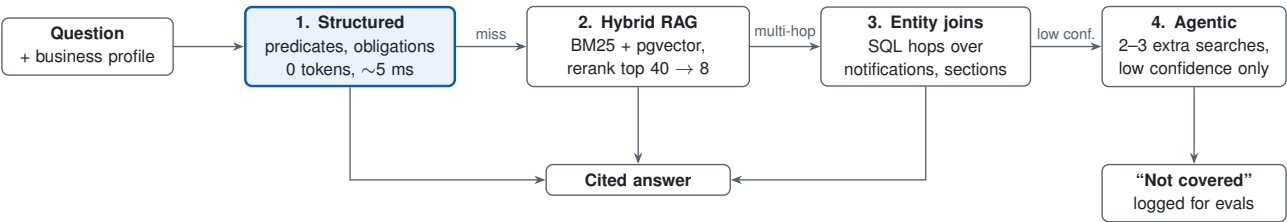
5.1 Models by task

Task	Model	Why, and fallback
Rule extraction	Frontier model (Claude Sonnet class), temperature 0, JSON schema enforced	Accuracy on long legal text with citations. Fallback: a second provider with the same schema
Free-text judgement	Small model (Claude Haiku class) with a short prompt; returns applies / not / unsure with confidence	Low volume, only for predicates with free text. Unsure goes to review
Grounded answers	Small model with a citation-required prompt	Interactive latency; escalates to the frontier model when confidence is low
Document classification	Fine-tuned DeBERTa-v3-base; LLM few-shot until 2,000 labels exist	Cheap and deterministic on every document
Embeddings	bge-m3 self-hosted, or Voyage in the MVP	Multilingual (English, Hindi), 8k context

Task	Model	Why, and fallback
Reranking	bge-reranker-v2-m3 cross-encoder	Large precision gain on legal clauses at low cost
OCR	PaddleOCR on the worker	Scanned notifications are common; data stays in region

5.2 Layered retrieval

Questions pass through progressively more expensive layers, and most stop early. This is the same pattern that lets SmartShelfKart answer 76% of benchmark questions without a model call.



- 1. **Structured first.** “Does e-invoicing apply to me?” or “when is my GSTR-3B due?” is answered from rule predicates and the business’s obligations in Postgres, with no model call.
- 2. **Hybrid RAG.** Chunks are clauses, not token windows, and each is embedded with a contextual header (“CGST Notification 17/2026 · e-invoicing · effective 1 Oct 2026”). Metadata filters apply first: `regulator`, `status = published`, and `effective_from ≤ as_of < effective_to`. BM25 and vector results are fused by reciprocal rank and reranked by the cross-encoder.
- 3. **Entity joins (SAG-style).** Each clause’s entities (notification numbers, sections, HSN codes, form names) sit in a `clause_entity` table. Multi-hop questions such as “what changed for GSTR-1 since April?” become SQL joins across documents that share entities, and the results are reranked like layer 2. This gives graph-style answers without building or maintaining a knowledge graph.
- 4. **Agentic fallback.** Only when the top rerank score is low, the model may issue two or three further searches before answering or refusing.

Supersession-aware answers. After retrieval, each hit is followed through the `supersedes` links to the version in force on the question’s date, so an answer never quotes a rule that has been replaced.

Answering with citations only. The prompt receives the selected clauses with their `clause_refs` and must cite by ref. The response is post-checked: every cited ref must be in the retrieved set, and the quoted span must appear in the clause (fuzzy match ≥ 0.85). Two failed checks produce “not covered”.

Relation to KAG. The design follows the principles of Knowledge Augmented Generation (Ant Group, 2024) without its graph engine: a schema-constrained vocabulary (the ontology), logic-first reasoning (three-valued predicates), and two-way links between facts and source text (citations from rule versions to clauses).

5.3 Extraction pipeline

- 1. The parser emits clauses with stable refs and table renderings.
- 2. The extractor prompt (versioned file, owned, tested) asks for a `RuleCandidate` JSON: title, summary, predicates over ontology attributes, obligation template, effective period, supersedes references and citations.
- 3. Validators run as a chain of responsibility: JSON schema, ontology attribute check, citation verification (each quote must appear in its clause), date sanity, forbidden fields.
- 4. A second pass critiques the candidate against the source and lowers confidence on disagreement.

5. The candidate enters the review queue. Every reviewer edit is stored as a diff and becomes an extraction eval case.

Regulator documents are untrusted input: instruction-like text inside a document is treated as data, and the output schema has no free-form action fields.

5.4 Evaluation

Metric	Definition	Gate ^T
Context recall	Share of gold clauses present in the retrieved set	≥ 0.92
Context precision	Share of retrieved clauses that are relevant	≥ 0.80
Grounded-answer rate	Answers whose every claim is supported by a cited clause (LLM judge + 10% human audit)	≥ 0.97
Citation correctness	Cited spans that match the clause text	≥ 0.98
Extraction acceptance	Candidates approved without edits	≥ 0.90
Applicability precision / recall	Against the labelled decision set	0.95 / 0.90
Refusal accuracy	Unsupported questions answered “not covered”	≥ 0.95
Deterministic coverage	Share of Q&A answered by layer 1	tracked
Cost per feature	Tokens and INR per extraction, decision, answer	budget

Golden sets start with 200 documents and 500 questions written by analysts, and grow from review edits, “not covered” questions and user thumbs-down after triage. The model call is scripted in CI so that everything downstream of it runs for real at zero token cost; a nightly run uses real providers.

Cost controls. Semantic cache in the gateway; small models for classification and judgement; the Batch API (half price) for backfills; deterministic predicates so most evaluations never call a model; per-tenant and per-feature budgets with alarms at 80%.

6 Data and APIs

6.1 Stores

One PostgreSQL cluster with a schema per service is the system of record through year 2. pgvector, full-text search and Redis cover search and caching until volumes justify dedicated engines.

Store	Used for	When to move
PostgreSQL 16 + pgvector (HNSW)	Transactions, row-level security, JSONB attributes, clause vectors	Split hot schemas past ~5,000 writes/s; Qdrant past ~50M vectors
Postgres full-text	BM25-style keyword retrieval	OpenSearch at 200+ sources or multilingual search
Redis 7	Sessions, rate limits, semantic cache keys, flag cache	—
Object store (S3/R2)	Raw regulator files, parsed artefacts, evidence uploads; versioned and append-only	—
Kafka (Redpanda locally)	Domain events, outbox relay, replay	—
Temporal	Pipeline and fan-out workflow state	—

6.2 Key tables

Table	Important columns	Indexes and rules
<code>regulatory.document</code>	<code>source_id</code> , <code>sha256</code> , <code>doc_type</code> , <code>published_at</code> , <code>raw_uri</code>	<code>unique(sha256)</code> ; append-only
<code>regulatory.clause</code>	<code>document_id</code> , <code>clause_ref</code> , <code>text</code> , <code>embedding vector(1024)</code>	HNSW(embedding); GIN(tsvector)
<code>regulatory.clause_entity</code>	<code>clause_id</code> , <code>entity_type</code> , <code>value</code>	(<code>entity_type</code> , <code>value</code>); (<code>clause_id</code>)
<code>regulatory.rule_version</code>	<code>rule_id</code> , <code>version</code> , <code>predicates jsonb</code> , <code>template jsonb</code> , <code>effective_from/to</code> , <code>status</code> , <code>supersedes uuid[]</code>	(<code>status</code> , <code>effective_from</code>); GIN(<code>predicates</code>); status changes by trigger
<code>regulatory.citation</code>	<code>rule_version_id</code> , <code>clause_id</code> , <code>quote</code> , <code>verified</code>	<code>verified</code> must be true to publish
<code>business.profile</code>	<code>tenant_id</code> , <code>attributes jsonb</code> , <code>version</code>	RLS on <code>tenant_id</code> ; history in <code>profile_version</code>
<code>work.applicability_decision</code>	<code>business_id</code> , <code>rule_version_id</code> , <code>result</code> , <code>confidence</code> , <code>evaluated jsonb</code> , <code>profile_version</code>	one row per evaluation, never updated
<code>work.obligation</code>	<code>tenant_id</code> , <code>business_id</code> , <code>rule_version_id</code> , <code>due_at</code> , <code>status</code>	<code>unique(business_id, rule_version_id)</code> ; RLS
<code>work.notification</code>	<code>obligation_id</code> , <code>channel</code> , <code>dedupe_key</code> , <code>status</code>	<code>unique(dedupe_key)</code>
<code>audit.event</code>	<code>actor</code> , <code>action</code> , <code>subject</code> , <code>before</code> , <code>after</code> , <code>at</code>	insert-only role; exported with object lock

Migrations are expand-and-contract: add, dual-write, backfill, switch reads, then drop in a later release. Embeddings carry the model name and version; a model change adds a column and re-embeds in the background.

6.3 Public REST API (v1)

OpenAPI 3.1, versioned under `/v1`, shared by the web app, the WhatsApp bot backend and partners. Conventions: OIDC bearer tokens for users and OAuth client credentials for partners; `Idempotency-Key` on every creating POST; cursor pagination; RFC 9457 problem+json errors with a correlation id; per-tenant rate limits; 90 days' notice for additive changes to partners.

Endpoint	Purpose
POST <code>/v1/businesses</code>	Create a profile; returns the onboarding checklist
PATCH <code>/v1/businesses/{id}</code>	Update attributes (validated against the ontology); triggers recompute
GET <code>/v1/businesses/{id}/obligations</code>	Obligations with citations, filterable by status and due date
POST <code>/v1/obligations/{id}/status</code>	Mark in progress, done or waived, with evidence ids
GET <code>/v1/changes?since=</code>	Published rule versions with plain-language summaries
GET <code>/v1/changes/{id}/impact</code>	For a CA tenant: which clients are affected and why
POST <code>/v1/qa</code>	Grounded answer with citations, confidence, or <code>not_covered</code>
POST <code>/v1/webhooks</code>	Register an endpoint for obligation and change events
GET <code>/v1/audit?subject=</code>	Audit trail for an obligation or profile

Webhooks are signed with HMAC-SHA256 over the body plus a timestamp, retried with backoff for 24 hours, then paused with the tenant notified; a replay endpoint re-sends by event id. Inside the cluster, reads are REST over mTLS and state changes travel only as events; the engine's single-business evaluation is also exposed over gRPC for the fan-out workers.

```

POST /v1/qa
{ "business_id": "biz_8f3", "question": "Do I need e-invoicing from October?", "as_of":
  "2026-10-01" }

200 OK
{ "status": "answered", "layer": "structured", "confidence": 0.97,
  "answer": "Yes. From 1 Oct 2026 e-invoicing applies because your turnover band is above the
    threshold.",
  "citations": [ { "rule_version_id": "rv_214", "clause_ref": "N-17/2026:2", "quote": "..." } ],
  "obligations": [ "obl_5521" ] }

```

6.4 First partner: SmartShelfKart

SmartShelfKart, the author's multi-tenant inventory platform, already holds most of what a profile needs. Through the partner API it pushes turnover band (computed from invoices), HSN codes and GST rates (from the product catalogue) and states (from warehouse locations), and receives `change.published` and `obligation.created` webhooks. A GST rate change on one HSN code can then alert exactly the shops that sell those products, and point to the open quotations still priced at the old rate.

7 Code design

7.1 Layers inside every service

Every Python service has the same four layers, and import-linter enforces the direction of dependencies in CI.

Layer	Contains	Rule
API	FastAPI routers, request/response schemas, auth dependencies	No business logic
Application	One use-case class per action or event handler; unit of work; outbox	Orchestrates; knows no SQL
Domain	Aggregates, value objects, domain services, events, repository protocols	Standard library only; unit-tested without a database
Infrastructure	SQLAlchemy repositories, Kafka, S3, HTTP and model-provider adapters	Implements protocols the domain declares

7.2 Core protocols (from the domain kernel)

```

class SourceAdapter(Protocol):
    def list_documents(self, since: datetime) -> Iterable[DiscoveredDocument]: ...
    def fetch(self, ref: DocumentRef) -> RawDocument: ...

class DocumentParser(Protocol):
    def supports(self, doc: RawDocument) -> bool: ...
    def parse(self, doc: RawDocument) -> ParsedDocument: ...

class RuleExtractor(Protocol):
    def extract(self, doc: ParsedDocument) -> RuleCandidate: ...

class PredicateEvaluator(Protocol):
    def evaluate(self, p: Predicate, profile: ProfileSnapshot) -> PredicateResult: ...

class LLMProvider(Protocol):
    def complete(self, req: CompletionRequest) -> CompletionResponse: ...

class VectorStore(Protocol):
    def search(self, q: Vector, f: ClauseFilter, k: int) -> list[ScoredClause]: ...

```

7.3 Patterns and where they are used

Pattern	Where, and why
Strategy + Registry	Source adapters, parsers, channels and model providers: a new regulator or provider is a registered class, not an edit
Specification	<code>Predicate</code> , <code>AllOf</code> , <code>AnyOf</code> , <code>Not</code> : applicability rules compose and are unit-tested
Repository + Unit of Work + Outbox	State change and its event commit in one transaction
State	Explicit transition tables for rule versions and obligations
Template Method	<code>PipelineStage</code> : load, process, validate, emit, with uniform tracing and retries
Chain of Responsibility	Validators on LLM output: schema, ontology, citation, forbidden fields, confidence
Decorator	Cache, retry and circuit breaker wrapped around <code>LLMProvider</code> and <code>SourceAdapter</code>
Anti-corruption layer	GSTIN lookup and WhatsApp payloads mapped to domain types at the boundary

SOLID in practice: one use case per class; adapters extend the system without editing it; every `SourceAdapter` passes the same contract tests; the engine depends on a narrow `RuleReader`, not the whole rulebook API; the domain declares protocols and a composition root per service wires them.

7.4 Repository layout

```

compliancewatch/
  apps/web/                Next.js: owner portal, CA dashboard, /admin tools
  apps/whatsapp-bot/       Webhook receiver and conversation state (TypeScript, Hono)
  services/<name>/          10 FastAPI services: api/ application/ domain/ infrastructure/
                           + migrations/ (own schema) + tests/unit|integration|contract
  services/pipeline/        adapters/ parsers/ prompts/ workflows/
  packages/contracts/       OpenAPI specs, event JSON Schemas, generated clients (py, ts)
  packages/domain-kernel    Shared value objects, predicate algebra, protocols
  packages/ontology/        GST attributes in YAML, loader, validator
  packages/py-common/       Settings, logging, tracing, health routes, app factory
  packages/ui/              Shared React components and tokens
  infra/                    terraform/ helm/ argocd/ dev/ (compose assets)
  evals/                    golden/ (extraction, qa, applicability) + harness/
  docs/                     adr/ runbooks/ onboarding/

```

Enforced by tooling. No imports across services, and no infrastructure imports in a domain layer (import-linter); strict types on both sides (mypy `–strict`, tsc `–strict`); an 80% coverage floor on domain and application layers; every event schema-checked for compatibility; every prompt registered with an owner and at least one eval case; every tenant table carries `tenant_id` and an RLS policy (migration lint); conventional commits; secrets scanned by gitleaks in pre-commit and CI.

7.5 Technology choices

Layer	Choice	Reason (alternative considered)
Services	Python 3.12, FastAPI, Pydantic v2	Best document and AI ecosystem; typed contracts and OpenAPI for free (Go, Django)
Workflows	Temporal	Durable retries and visibility for long, resumable work (Celery, Airflow)
Events	Kafka + outbox	Ordered and replayable per consumer group (RabbitMQ, SQS)
Database	PostgreSQL + pgvector	Transactions, RLS, JSONB, vectors and full-text in one engine (separate vector DB)

Layer	Choice	Reason (alternative considered)
Web	Next.js, React, Tailwind	One stack for product and internal tools (Remix, Retool)
Identity	Keycloak	OIDC and SAML, data stays in India, no per-user cost (Auth0, Clerk)
LLM access	Own gateway	Central cost ledger, caching, prompt versions, PII control; no LangChain in production so retrieval stays visible to evals
Monorepo	uv workspaces; pnpm + Turborepo	Fast installs, affected-only builds (Nx, Bazel)
Observability	OpenTelemetry, Prometheus, Grafana, Langfuse	Open standards; LLM traces with prompt version and cost (Datadog)

7.6 Ownership

Teams meet at contracts (OpenAPI, event schemas, the ontology), not at shared code. `CODEOWNERS` maps every directory to one team; until the teams are staffed, two combined teams cover them.

Team	Owns
Regulatory Intelligence	pipeline, rulebook, review workbench
AI Platform	llm-gateway, eval, qa, LLM judgement in the engine, prompts, embeddings
Core Product	profile, obligation, notification, engine fan-out, web and WhatsApp apps, public API
Platform	infra, CI/CD, observability, py-common, domain kernel (custodian)
Identity and Partner	identity, API keys, webhooks, admin console, billing
Regulatory analysts	Ontology content, golden sets, review queue, customer-facing wording

8 Security, deployment and operations

8.1 Security and privacy

A compliance product that leaks a client list or sends a wrong obligation loses its customers, so tenant isolation, an immutable audit trail and treating regulator documents as untrusted input are built in from the first phase.

Threat	Control
Cross-tenant access through a bug	Postgres RLS keyed on the token's tenant claim; tenant checked at the gateway and again in each service; tests attempt cross-tenant reads on every endpoint
Prompt injection via documents or uploads	Documents are data; no tool calls from document content; output schemas have no action fields; human review before publish
Wrong obligation sent at scale	Two-person approval for high-impact rules; dry-run impact counts before publish; pausable fan-out; one-click rollback closes obligations with a reason and sends a correction
Credential leakage	Secrets manager with rotation; short-lived tokens; gitleaks; partner keys hashed and shown once
Supply chain	Locked dependencies, SBOM per image, Trivy scan gating deploy, signed images
Model provider exposure	PII scrubbed by the gateway; zero-retention settings where offered; no customer data used for training
Insider misuse	Read-only by default; reason-required, time-boxed impersonation; every admin action audited

DPDP Act 2023. A data map gives every personal field a purpose and a retention period. Consent is recorded with the notice version. Export and deletion are served within 30 days through a `tenant.deletion.requested`

event that cascades across services. All customer data stays in India. Every obligation shows its cited clause and a “reviewed by” line, and the terms state that the service monitors compliance and is not legal advice.

8.2 Deployment profiles

The same containers run in three profiles. Every external dependency sits behind an interface, so moving between profiles swaps implementations and rewrites nothing.

Layer	Local (development)	MVP (Phases 1–2)	Scale (Phase 3+)
Web	<code>next dev</code>	Vercel	Vercel or EKS
Services	<code>make run</code> (uvicorn)	One modular deployable on Railway, Render or Fly.io	10 services on EKS, Helm, Argo CD canaries
Workflows	Temporal in Docker	Temporal Cloud or Inngest	Self-hosted Temporal
Events	Redpanda in Docker	Postgres outbox + managed queue	Amazon MSK
Database	Postgres + pgvector in Docker	Neon or Supabase	Aurora PostgreSQL, multi-AZ
Embeddings	bge-m3 on CPU or Voyage	Voyage API	bge-m3 on a GPU pool
Public URL	Cloudflare Tunnel	Platform domains	ALB + WAF

CI/CD. Affected-only GitHub Actions: lint and type-check; unit tests with the coverage gate; contract checks (OpenAPI diff, event schema compatibility, import-linter, migration lint); integration tests on testcontainers; the eval suite whenever prompts, retrieval or the engine change; image build with SBOM, Trivy scan and signature. Merges to `main` deploy to staging; production releases go out as a 5%, 25%, 100% canary with automatic rollback on error rate, latency or, for the engine, decision-flip rate against the previous version.

8.3 Service levels and alerts

Flow	Indicator	Objective ^T
Public API	Availability; read latency p95	99.9%; < 300 ms
Grounded Q&A	Latency p95; grounded-answer rate	< 4 s; ≥ 97%
Pipeline freshness	Source publish to review queue	95% within 6 h
Fan-out	Publish to last obligation, 100k businesses	< 60 min
Notifications	Delivered within 15 min; duplicates	99%; 0
Durability	Lost regulator documents	0

Alerts that page: a source silent for two expected cadences; dead-letter depth above zero for 30 minutes; decision-flip rate above 2% after a deploy; SLO burn rate above 2× over an hour; any duplicate notification; LLM spend past 80% of budget before the 20th. Every alert links to a runbook, and rule publication pauses automatically while a severity-1 incident is open.

8.4 Testing

Level	Coverage
Unit and property	Domain and application with fakes; Hypothesis property tests for the predicate algebra, periods and dedupe keys
Contract	Every endpoint against its OpenAPI spec (schemathesis); every event against its schema; consumer-driven tests between teams
Integration	Repositories, outbox relay, consumers and Temporal workflows on testcontainers

Level	Coverage
Adapter conformance	Every source adapter and parser against recorded fixtures per regulator
AI evals	Golden sets and the gates in Section 5.2; scripted model in CI, real providers nightly
End to end	Playwright: onboard, receive a change, ask a question, close an obligation, CA bulk flow
Load and chaos	k6 fan-out of one rule to 100k synthetic businesses; pod kills, broker loss, provider timeouts
Data quality	Nightly rulebook invariants: citations resolve, dates ordered, no supersession cycles

8.5 Running cost

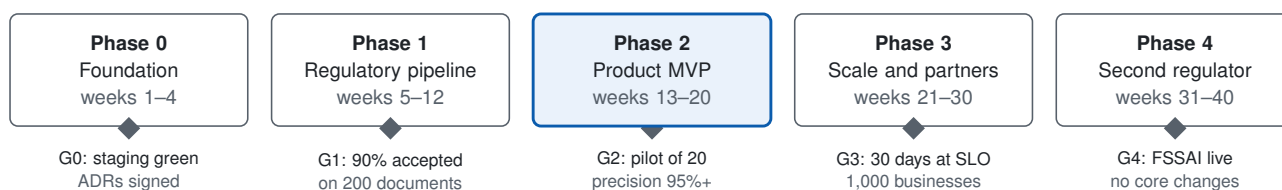
Estimates from public prices in September 2026, converted at about INR 88 per USD. The fixed platform cost dominates at small scale; LLM cost grows with documents and questions, not with the number of businesses.

Scenario	Monthly (INR)	Main items
Local machine, 10 businesses	1,000–1,700	Claude API, WhatsApp messages, electricity, domain
MVP hosting, 10 businesses	2,500–4,500	Vercel Pro, Railway, Neon, Claude API; free tiers elsewhere
MVP hosting, 1,000 businesses	15,000–25,000	Adds Q&A and judgement tokens ($\approx$ INR 6 per business) and WhatsApp
Scale profile on AWS Mumbai	2.4–4 lakh	EKS, GPU pool, Aurora, MSK, staging; justified only at thousands of paying businesses

9 Roadmap, decisions and risks

9.1 Roadmap

Forty weeks in five phases. Each phase ends at a gate with a number attached, and the next phase starts only when the gate is passed. The Phase 2 gate, a pilot with real businesses, is the one that proves the product.



- Foundation.** Monorepo, service template, CI gates, dev stack, domain kernel, ontology v0, ADRs 1–6. *(Done.)*
- Regulatory pipeline.** CBIC adapters, parser with OCR, extractor with citation checks, review workbench, versioned rulebook, backfill of about 200 documents, first golden sets.
- Product MVP.** Onboarding, profile, applicability engine, obligations, WhatsApp and email, layered Q&A, basic CA view; pilot with 20 businesses and 3 CA firms, including SmartShelfKart customers.
- Scale and partners.** CA dashboard and bulk actions, partner API and webhooks, evidence uploads, eval and cost dashboards, SLO tooling, penetration test.
- Second regulator.** FSSAI through the extension points only, Hindi UI, SOC 2 readiness, disaster-recovery drill.

9.2 Decision log

ADR	Decision	Status
001	Monorepo, one directory per service, shared contracts package	Accepted
002	Python and FastAPI for services; Next.js and TypeScript for apps	Accepted
003	PostgreSQL with pgvector as system of record and vector store until Phase 3	Accepted
004	Temporal for pipeline and fan-out orchestration	Accepted
005	Kafka with a transactional outbox for all domain events	Accepted
006	No rule published without human approval; two-person rule for high-impact	Accepted
007	Deterministic predicates first; LLM judgement only for free-text conditions	Accepted
008	Own LLM gateway; no LangChain or LlamaIndex in production	Accepted
009	Keycloak self-hosted for identity	Proposed
010	OpenSearch in Phase 3 for keyword search at scale	Proposed
011	India first, GST launch regulator, FSSAI second	Proposed
012	Layered retrieval: structured, hybrid RAG, SQL entity joins, agentic fallback	Proposed
013	Managed MVP deployment profile before the Kubernetes profile	Proposed

9.3 Risks

Risk	Impact	Mitigation in the design
Regulator sites change layout or block crawlers	Medium	Adapter fixtures catch breakage in CI; freshness alerts; manual upload path; Gazette as a second source
Extraction quality below the Phase 1 gate	High	Human review so errors never reach customers; extract-then-critique; golden sets grow from every edit
Wrong obligation sent to many businesses	High	Two-person approval; dry-run impact counts; pause and rollback; correction notices
Ontology becomes a bottleneck	Medium	Versioned package, editor tool with usage counts, monthly release train
LLM cost above budget	Medium	Gateway budgets, caching, small models, deterministic answers first
Model provider outage or policy change	Medium	Second provider warm with identical schemas; self-hostable embeddings
Liability for an incorrect obligation	High	Citations and reviewed-by line; “monitoring, not advice” terms; error-report flow; indemnity insurance

9.4 Glossary

Ontology The shared vocabulary of business attributes that rules and profiles are written in.

Predicate A condition over ontology attributes attached to a rule version.

Rule version One dated, cited, approved statement of a rule.

Applicability decision

The stored result of evaluating a rule version against one profile version.

Obligation The dated task a decision creates for a business.

Outbox A table written in the same transaction as a state change, relayed to Kafka.

Gate A measurable exit criterion for a roadmap phase.

References: Ant Group, *KAG: Boosting LLMs in Professional Domains via Knowledge Augmented Generation*, [arXiv:2409.13731](https://arxiv.org/abs/2409.13731); SAG: *SQL-Retrieval Augmented Generation with Query-Time Dynamic Hyperedges*, [arXiv:2606.15971](https://arxiv.org/abs/2606.15971).